

\$0 Startup: Launching a Full Tech Product on a Shoestring Budget

What if i told you that for less than the price of coffee, i build a fully functional tech product “a deep link service”.

For those who want to know more on deep links please go through your search engine and you will find tons of references.

The concept was simple yet ambitious: a service that creates intelligent deep links that work across all platforms and handle edge cases automatically with a cross platform support like android, ios and pc.

In a world where tech products often come with hefty price tags, I set out to prove that innovation doesn't have to break the bank. My goal? Build a functional tech product for just \$2.

The challenge was on.

What is Deep link

A deep link is a replacement link, similar to a URL shorter for another link. But it behaves differently based on the platform it is opened on. The behavior can be programmed by you.

With deep links, your users get the best available experience for the platform they open your link on. If a user opens a deep Link on iOS or Android, they can be taken directly to the linked content in your native app. If a user opens the same deep link in a desktop browser, they can be taken to the equivalent content on your website.

In addition, deep links work across app installs: if a user opens a deep link on iOS or Android and doesn't have your app installed, the user can be prompted to install it; then, after installation, your app starts and can access the link [[source](#)].

Challenges

1. [Cheapest Domain](#)
2. [Cost Effective DNS management system](#)
3. [Choosing the tech stacks](#)
4. [Application Architecture](#)
5. [Free/Cheapest Hosting](#)
6. [SSL](#)

Finding the cheapest domain

I wasn't about to blow my budget on a fancy ".com" domain. Instead, I set out on a mission to find the cheapest yet reasonable domain name across various providers like GoDaddy and Namecheap. The challenge? It had to be both affordable and practical.

After extensive research, I found [dynamc.link](#) on [Spaceship](#) for just **\$2**—a perfect fit for my budget!

The Payment Struggle

Reserving the domain was easy, but making the payment? Not so much. My biggest hurdle was that Nepali bank-issued dollar cards weren't recognized by international payment gateways. I reached out to friends who had dollar cards, but every attempt failed.

In the end, I turned to a friend in Australia, who kindly made the payment on my behalf. And that's how I finally secured [dynamc.link](#)—the foundation of my tech product, built on just \$2!

Perfect DNS Manager

DNS might sound boring—until you realize it's the backbone of your app. I wasn't about to spend extra, so I went with [Cloudflare's free tier](#). It's fast, reliable, and does the job without sneaky upsells.

The best part? It takes care of **SSL for all my subdomains** effortlessly. A huge shoutout to Cloudflare for making this easy—especially since it played a key role in my hosting trick later.

Cost: \$0

Choosing the Tech Stacks

When building a product on a tight budget, choosing the right tech stack is crucial. I needed something **scalable, widely used, and backed by a strong community and should be free**—so I went with:

- **Frontend: React** with **Material Design** – A powerful combination for creating modern, responsive UIs with minimal effort.
- **Backend: Django** – A battle-tested framework that provides rapid development, built-in security, and a structured approach.
- **Database: PostgreSQL** – A highly scalable and reliable database, perfect for handling growing data needs.
- **Containerization: Docker** – Ensures a smooth deployment process, making the app easy to scale and manage across different environments.

These technologies weren't just picked at random; they are **popular, well-documented, and widely supported**—making development smoother while ensuring long-term scalability.

Cost: \$0 (Thanks to open-source!) 🚀

Application Architecture

To ensure scalability and maintainability, I adopted a **microservices architecture**, splitting the backend into two independent services:

1. **Deep Link Resolver** – This service is responsible for resolving shortened links and redirecting users to the appropriate destination. It ensures fast and efficient link resolution.
2. **API Service** – This handles all business logic, user requests, and interactions with the database, ensuring smooth communication between the frontend and backend.

Both services are built using **Django**, containerized with **Docker**, and communicate seamlessly with **PostgreSQL** as the central database. This separation allows for better **scalability, fault isolation, and easier updates**.

On the frontend, **React with Material Design** ensures a sleek and responsive user experience, while **Cloudflare** provides **DNS management and free SSL** to secure all requests.

This architecture ensures that each component can scale independently, making it efficient and future-proof. 🚀

Hosting

Hosting is often the biggest challenge when building a product on a budget. Popular cloud platforms like **AWS, GCP, and Azure** offer powerful infrastructure, but their pricing can quickly become expensive—even for small-scale projects.

Home Server with Cloudflare Zero Trust

To cut costs, I found a creative solution: **Cloudflare Zero Trust**. Instead of paying for traditional hosting, I used it to **tunnel TCP requests to my old PC**, which had an **Intel i3 processor and 4GB RAM**—a machine that had been collecting dust.

I installed **Ubuntu Server OS** and turned it into a **home server**, where I hosted:

- The **web frontend** (React)
- The **backend API** (Django)
- The **PostgreSQL database**

This setup worked well, but due to the **limited resources** of my old PC, I couldn't accommodate one critical service—the **Deep Link Resolver**.

Finding a Free VPS for the Link Resolver

I started looking for **free-tier VPS providers** and discovered that **AWS offers a 12-month free EC2 instance for new users**. While it's minimal, it was perfect for handling the **Deep Link Resolver**, which required low resources but needed to be **accessible with low latency**.

By strategically splitting services between my **home lab** and a **free AWS instance**, I managed to get hosting at **zero cost**—proving that with the right tools, you don't need deep pockets to launch a tech product! 🚀

Managing SSL for custom domains

As my platform started to grow, I realized that some clients might want to connect their own **custom domain** instead of using the free subdomain I provided. That meant my system needed a way to **manage SSL certificates** for those custom domains automatically — because every website today needs to be secure and accessible via HTTPS.

At first, I used **Nginx** with **Certbot** to handle SSL. It worked perfectly for a few users, but there was one major issue — **Certbot certificates expire every 90 days**. That meant I had to manually renew SSL certificates for each client every three months. It was manageable at first, but I quickly realized that as more clients started using their own domains, this manual renewal process would become a nightmare.

So I started looking for a smarter, hands-free solution. That's when I discovered **Caddy Server** — an open-source alternative to Nginx that automatically handles SSL certificates out of the box. Caddy doesn't just issue certificates; it also **renews them automatically before expiry**, saving me hours of repetitive work.

The best part? Caddy exposes an **API and a built-in dashboard**, which means I can add or remove domains programmatically — something Nginx simply couldn't do natively.

Although I've always been a big fan of Nginx, Caddy turned out to be a real **boon for solo founders** like me. It simplified SSL management to the point where I never had to think about renewals again. So, I migrated from Nginx to Caddy — and that small decision saved me countless hours of maintenance time while keeping my zero-dollar setup fully automated.

Here's A GitHub Repo For Caddy Server My Setup: <https://github.com/hvo7mdq/caddy>

The Finish Line

Building a tech product with just **\$2** seemed impossible at first, but with the right strategy, open-source tools, and creative problem-solving, this made it happen.

From finding the **cheapest domain** to setting up **free hosting**, I carefully picked every component to ensure **scalability, reliability, and cost-efficiency**. With **React, Django, PostgreSQL, Docker, and Cloudflare Zero Trust, Caddy** I built a fully functional system while keeping costs near zero.

This journey proved that **budget constraints shouldn't stop innovation**. With resourcefulness and the right tech stack, anyone can launch their own product—even with just a couple of dollars! 🚀

This is just the beginning. Now, it's time to scale and take it to the next level!